

IOC ANALYZER

Administrator & Engineer Manual

Complete reference: features, deployment, and operations
v2.0 · 2025

Threat Intelligence & Incident Response Platform

1. Introduction

IOC Analyzer is a desktop threat intelligence and incident-response platform for Windows. It enables security analysts, administrators, and engineers to detect malware artifacts, scan files with YARA rules, check file hashes against VirusTotal, collect network intelligence, and scan multiple hosts remotely from a single workstation.

1.1 System Requirements

- OS: Windows 10 / 11 (64-bit)
- Python 3.11+ with PyQt6, requests, psutil
- Network access for API calls (VirusTotal, AbuseIPDB, Groq/Claude)
- yara64.exe OR yara-python for YARA scanning
- pywinrm for automated remote agent deployment (optional)

1.2 First Launch

Run the application from the project root:

```
python main.py
```

On first launch, config.json is created automatically. Open Settings to enter your API keys (VirusTotal, AbuseIPDB, Groq or Claude).

2. Feature Reference

2.1 Dashboard

The Dashboard provides a real-time overview of scanning activity. It displays counters for YARA hits, scanned hashes, network lookups, and a live event feed that records every significant finding across all tabs.

2.2 Hash Lookup

Enter one or more SHA-256/MD5/SHA-1 file hashes to query the VirusTotal API. Results show the detection ratio, file name, file type, and a direct link to the VirusTotal report. The free VT API supports 4 requests/minute; configure the delay in Settings.

2.3 IOC Collection

Collect and manage Indicators of Compromise from the local host: running processes, active network connections, and autorun registry entries. Suspicious processes (outside C:\Windows and C:\Program Files) are highlighted automatically.

2.4 YARA Scanner

Scan a file or directory using YARA rules. The scanner ships with 40+ built-in rules covering ransomware, RATs, credential stealers, C2 frameworks, and offensive tools. You can also load custom .yar files or write rules in the built-in editor.

- Select one or more rules (or 'Select all') from the left panel.
- Choose a target file or folder using the File / Folder buttons.
- Click Scan YARA. Results appear in the table with severity colour coding.

! YARA scanning requires either yara64.exe in the project folder or yara-python installed.

2.5 Network Intel

Look up IP addresses and domains against AbuseIPDB and VirusTotal. Results include abuse confidence score, country, ISP, and threat categories.

2.6 AI Assistant

Chat with an AI model (Grok / Claude) for threat analysis. Paste IOC data, log snippets, or YARA rule code to get explanations, MITRE ATT&CK mapping, and remediation recommendations. Configure the provider and API key in Settings.

2.7 Report Builder

Generate incident reports in PDF or HTML format. Reports aggregate findings from all tabs, include timestamps, severity levels, and recommendations. Enable auto-save in Settings to export a report after every scan automatically.

2.8 Memory Scanner

List all running processes and scan a selected process's executable image with YARA rules. Useful for detecting in-memory malware that has not written files to disk. Requires elevated privileges for some processes.

2.9 Quarantine

Move suspicious files to an isolated quarantine folder. Quarantined files are renamed to prevent accidental execution. You can restore or permanently delete them from this tab.

2.10 Settings

Configure the application:

- API keys for VirusTotal, AbuseIPDB, Groq, Claude
- Default folders for results and quarantine
- VirusTotal request rate limit
- Auto-save reports toggle
- Interface language: ENG / RUS / ҚАЗ

2.11 Hosts (Multi-Host Scanning)

The Hosts tab lets you manage and scan multiple Windows machines from a single IOC Analyzer instance. Each remote host runs a lightweight Flask HTTPS agent (agent.exe) that exposes scanning endpoints over port 5555.

- Add a host: enter its IP, port (5555), and API token.
- Ping: verify connectivity to the agent.
- Scan: run YARA, IOC collection, and/or file hash scanning on the remote host.
- Deploy: automatically install the agent via WinRM (requires WinRM enabled).

The host selector dropdown in the main header shows the currently active host. Selecting a host here makes it the default for all scanning operations.

3. Deploying a Remote Agent — Step by Step

3.1 What the Agent Does

agent.py is a self-contained Flask HTTPS server that runs as a Windows Service on each monitored host. It exposes six scanning endpoints protected by an API token. All traffic is encrypted with a self-signed TLS certificate generated automatically on first run.

- GET /ping — returns hostname, OS, agent version
- GET /info — returns CPU/RAM/disk usage and logged-on users
- POST /scan/yara — runs YARA rules against a path
- POST /scan/ioc — collects processes, connections, autoruns
- POST /scan/memory — scans a process executable with YARA
- POST /scan/ hashes — returns SHA-256 hashes of files in a path

3.2 Method A — Manual Deployment

Use this method when WinRM is not available on the target host.

Step 1 — Install Python on the target host

```
winget install Python.Python.3.12
```

Step 2 — Copy the agent folder to the target host

Copy the agent/ directory to C:\IOCAgent\ on the remote machine.

```
# From your workstation:
copy agent\agent.py \\TARGET-HOST\c$\IOCAgent\
# Or use USB / shared folder
```

Step 3 — Install agent dependencies on the target

```
pip install flask psutil cryptography pywin32
```

Step 4 — Run the agent in foreground to generate token

```
cd C:\IOCAgent
python agent.py
# Output:
# [IOCAgent] Generating self-signed certificate...
# [IOCAgent] v1.0.0 starting on https://0.0.0.0:5555
# [IOCAgent] Token: a3f7c9d2... (32-char hex)
```

OK Copy the token — you will need it in Step 7. It is also saved in C:\IOCAgent\token.txt.

Step 5 — Install as Windows Service (optional but recommended)

```
# Run as Administrator:
python agent.py --install
python agent.py --start
# Or use: net start IOCAgent
```

Step 6 — Allow port 5555 in Windows Firewall

```
netsh advfirewall firewall add rule name="IOCAgent" dir=in action=allow protocol=TCP localport=5555
```

Step 7 — Add the host in IOC Analyzer

Open the Hosts tab, click '+ Add', and fill in:

- Name: any label (e.g. SRV-DC01)
- IP: the target host IP address
- Port: 5555
- Token: the value from token.txt

Step 8 — Test connectivity

Click the `Ping` button. The status should turn green (● online).

Step 9 — Run a scan

Select YARA and/or IOC checkboxes, set a scan path (e.g. C:\Users), and click `Scan`. Results appear in the table below.

3.3 Method B — Automated Deployment via UI

If the target host has WinRM enabled (port 5985), you can deploy the agent directly from the IOC Analyzer UI without touching the remote machine manually.

Prerequisites on the target host

```
# Run as Administrator on the TARGET host:
winrm quickconfig
Set-Item WSMAN:\localhost\Service\AllowUnencrypted -Value true
Set-Item WSMAN:\localhost\Service\Auth\Basic -Value true
# Or for NTLM (domain environment):
Set-Item WSMAN:\localhost\Service\Auth\NTLM -Value true
```

! WinRM is disabled by default on Windows workstations. The commands above must be run with admin rights.

Build agent.exe first (run once on your workstation)

```
cd agent
build.bat
# Produces: agent\dist\agent.exe
```

Deploy from UI

- Open the `Hosts` tab.
- Click `Deploy Agent`.
- Enter the target host IP, administrator username, and password.
- Click `OK` — IOC Analyzer copies agent.exe, installs the service, and retrieves the token automatically.
- Click '+ Add' and paste the displayed token to register the host.

3.4 Verifying the Agent is Running

From a browser or PowerShell on the local network:

```
$tok = Get-Content C:\IOCAgent\token.txt
Invoke-WebRequest -Uri https://TARGET-IP:5555/ping \
-headers @{ "X-Api-Token"=$tok } -SkipCertificateCheck
```

Expected response:

```
{"status": "ok", "hostname": "TARGET-HOST", "os": "Windows 11", "agent_version": "1.0.0"}
```

4. Security Notes

- The API token is a 64-character random hex string generated once at installation.
- All agent traffic is encrypted with TLS (self-signed certificate). `verify=False` is used by the client since the cert is self-signed — this is acceptable for isolated internal networks.
- The token is stored in plain text in `token.txt` on the agent host. Protect this file with filesystem ACLs.
- WinRM credentials are not stored by the application. They are used only during the Deploy operation and then discarded.
- `config.json` contains API keys in plain text. Do not commit it to version control or share it.
- The agent runs as a Windows Service under SYSTEM or a dedicated service account. Restrict access to `C:\Program Files\IOCAgent\`.

5. Troubleshooting

Agent shows 'offline' after ping

- Verify `agent.exe` / `agent.py` is running on the remote host.
- Check that port 5555 is open in the firewall: `netstat -ano | findstr 5555`
- Confirm the token in `hosts.json` matches `token.txt` on the remote host.

YARA scan returns `COMPILE_ERR`

- The rule contains a syntax error. Open the Rule Editor and check the rule text.
- YARA 4.5.5+ treats unreferenced strings as errors. Ensure every declared string is used in the condition.

VirusTotal returns 'Quota exceeded'

- The free VT API is limited to 4 requests/minute. Increase the rate limit delay in Settings.
- Check that your API key is valid at virustotal.com.

Deploy fails with 'pywinrm not installed'

```
pip install pywinrm
```

Deploy fails with WinRM connection error

- Ensure WinRM is enabled on the target: `winrm quickconfig`
- Check firewall rules for port 5985 on the target host.
- Try pinging the target to verify basic connectivity.